

UNIT 3

INPUT/OUTPUT HANDLING IN C++



Q.1: DESCRIBE BASIC STRUCTURE OF C++ PROGRAM

BASIC STRUCTURE OF C++ PROGRAM

C++ program is mainly divided in three parts:

1. Pre-processor Directives
2. Main Function Header
3. Body of program / Function

BASIC STRUCTURE OF C++ PROGRAM IS GIVEN BELOW

1. `#include<iostream>`
2. `using namespace std;`
3. `int main()`
4. `{`
5. statements;
6. `return 0;`
7. `}`

Q.2: DESCRIBE ELEMENTS OF BASIC STRUCTURE.

EXPLANATION OF BASIC STRUCTURE

• `#include<iostream>`

The statement starts with `#` symbol is called pre-processor directives. This statement tells the preprocessor to include the contents of `iostream` header file in the program before compilation. This file is required for input-output statements.

• `using namespace std;`

This statement is used to instruct the compiler to use standard namespace. A namespace is a declarative location that provides a scope to the identifiers. Namespace `std` contains all the classes, objects and functions of the standard C++ library.

• `int main()`

This statement is a function and used for the execution of C++ program. `int` means it returns integer type value.

• `{`

This symbol represents the start of main function.

• `statements;`

Statements are instructions that performs particular task. Statement terminator (`;`) is used to end every statement in C++ program.

• `return 0;`

This statement is used to return the value to the operating system. By default, main function returns integer value 0.

• `}`

This symbol represents the end of main function.

Q.3: DEFINE COMMENTS IN C++. ALSO DEFINE IT'S TYPES.

COMMENTS IN C++

Comments are special remarks that helps to understand different parts of the code in C++ program. Comments are ignored by the compiler. In C++,there are two types of comments statement.

1. Single line Comment
2. Multi line Comment

SINGLE LINE COMMENT

This type is used to write single line comment. Double slash (//) symbol of each single line comment. is used at the start

Example:

```
// This is my first C++ program
// This program displays a statement on screen
#include<iostream>
int main()
{
    puts("My First C++ Program");
    return 0;
}
```

Q.4: DEFINE MULTI COMMENTS IN C++

MULTI LINE COMMENT

This type is used to write multi line comment. Symbols (/* and */) are used at the start and end of comment statements.

Example:

```
/* This is my first C++ program
This program displays a statement on screen */
#include<iostream>
int main()
{
    puts("My First C++ Program");
    return 0;
}
```

Q.5: WHAT DO YOU MEAN BY INPUT/OUTPUT STATEMENT IN C++?

INPUT/OUTPUT STATEMENTS IN C++

Input and output statements are used to perform input & output operations in C++. These input / output statements are stored in header files like <iostream>. At the beginning of every program these header files must be included.

Q.6: DEFINE OUTPUT FUNCTION OR STATEMENT IN C++. DEFINE COUT STATEMENT AND PUT STATEMENT WITH SYNTAX AND EXAMPLE OF C++.

OUTPUT FUNCTION/STATEMENTS IN C++

cout STATEMENT

cout stands for "Character Output". In C++, cout sends formatted output to standard output devices, such as the screen. cout object is used along with the insertion operator (<<) for displaying output.

syntax

cont << variable; or cont << exp. / string;
cont << "This is C++ Programming";

Example:

```
cont << num1;
```

puts() STATEMENT

This function is used to print the string output. After printing the screen new line is automatically inserted.

syntax

```
puts("string constant");
```

Example:

```
puts("This is C++ Programming");
```

Q.7: DEFINE INPUT FUNCTION OR STATEMENT IN C++.

Following are the input functions/statement in C++.

- cin STATEMENT
- getchar() STATEMENT
- getch() STATEMENT
- getche() STATEMENT
- gets() STATEMENT

Q.8: DEFINE CIN STATEMENT WITH SYNTAX AND EXAMPLE OF C++.

CIN STATEMENT

cin stands for "Character Input". In C++, cin reads formatted data as input from keyboard. cin object is used along with the extraction operator (>>) to accept data from standard input device.

syntax

```
cin >> variable;
```

Example:

```
#include<iostream>
using namespace std;
int main()
{
    int b;
    cin >> b; // cin takes input in "b" variable
    return 0;
}
```

Q.9: DEFINE GETCHAR() STATEMENT WITH AN EXAMPLE OF C++.

GETCH() STATEMENT

The getch() function reads the available character from the keyboard. This function reads only single character at a time. When the user presses the key, getch() requires Enter key to be pressed. This function is defined in <stdio.h> header file.

Example:

```
#include<iostream>
#include<stdio.h>
```

```
using namespace std;
int main()
char b;
cout<<"\n Enter a character:";
b = getchar();
cout<<"\n Input character is " << b;
return 0;
```

Q.10: DEFINE GETCH() STATEMENT WITH EXAMPLE OF C++.**GETCH() STATEMENT**

The getch() function reads the character from the keyboard. This function reads only single character and not printed on the screen. This function takes input and does not requires Enter key to be pressed. This function is defined in <conio.h> header file.

Example:

```
char ch;
cout<<"\n Enter a character:";
ch = getch();
cout<<"\n Input character is " << ch;
return 0;
char name [25];
cout<<"\n Enter your name:";
gets(name);
cout<<"\n Your Name is " << name;
return 0;
using namespace std;
```

Q.11: DEFINE GETCHE() STATEMENT WITH EXAMPLE OF C++.**GETCHE() STATEMENT**

The getche() function reads the character from the keyboard and echoes on the screen. It reads only single character and displays on the screen. This function takes input and does not require Enter key to be pressed. This function is defined in <conio.h> header file.

Example:

```
#include<iostream>
#include<conio.h>
using namespace std;
int main()
char ch;
cout<<"\n Enter a character:";
ch = getche();
cout<<"\n Input character is " << ch;
return 0;
}
```

Q.12: DEFINE GETS() WITH SYNTAX AND STATEMENT WITH EXAMPLE OF C++.**GETS() STATEMENT****Example in C++.**

This function is used to reads characters or the string input and stores them until a newline character found. This function is defined in <stdio.h> header file.

Syntax

Gets("variable");

Example

Q.13: WHAT IS STATEMENT TERMINATOR.

STATEMENT TERMINATOR (;)

In C++, statement terminator is used to end the statement. Statements are terminated with semicolon (;) symbol. Every statement in C++ must be terminated otherwise and error message will occur.

Q.14: WRITE DOWN IN DETAIL ABOUT ESCAPE SEQUENCES IN C++.

ESCAPE SEQUENCES

Escape sequences are used to control the cursor moves on screen by using special codes. An escape sequence is a special non-printing characters consists of the escape character (the backslash "\") and a second (code) character. The list of the escape sequences is given below:

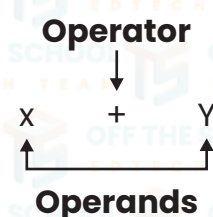
Escape Sequence	Compiler
\a	"a" means Alert or alarm. It causes a beep sound in the computer. Example: cout << "\a";
\b	"b" stands for backspace. It moves the cursor backspace. Example: cout << "\b";
\t	"t" stands for Horizontal tab. It is used to shift the cursor to a couple of spaces to the right in the same line. Example: cout << "\t";
\n	"n" stands for New line or line feed. It inserts a new line and cursor moves to the beginning of the next line. Example: cout << "\n";
\r	Carriage Return "r". It is used to position the cursor to the beginning of the current line. Example: cout << "\r";
\\	"backslash" is used to print the backslash character. Example: cout << "\\t";
\'	"Single quotation" It is used to print the 'apostrophe' (') sign or character. Example: cout << "\'";
\"	"Double quotation" It is used to print the quotation mark. Example: cout << "\"";

Q.15: WHAT ARE OPERATORS? WRITE DOWN NAMES OF OPERATORS IN C++.

OPERATORS

Operators are special symbols used for specific purposes. Operators perform mathematical operations on operands. For example: $x + y$. Here "x" and "y" are operands and "+" is the operator. There are seven types of operators that are used in C++ programming.

1. Arithmetic Operators
2. Increment Operators
3. Decrement Operators
4. Relational Operators
5. Logical Operators
6. Assignment Operators
7. Arithmetic Assignment Operators



Q.16: WHAT ARE ARITHMETIC OPERATORS?**ARITHMETIC OPERATORS**

Arithmetic operators are used to perform mathematical operations. All operators used integer & floating-point data type except remainder or modulus operator.

• **Addition (+):** It is used to perform arithmetic addition.

Example: $a + b$;

• **Subtraction (-):** It is used to perform arithmetic subtraction.

Example: $a - b$;

• **Multiplication (*):** It is used to perform arithmetic multiplication.

Example: $a * b$;

• **Division (/):** It performs the arithmetic division of two numbers.

Example: a / b ;

• **Remainder / Modulus (%):** It is used to find the remainder of a division. It returns the remainder of an integer value. The remainder operator is also known as the modulus operator. This operator is used only with integral data types.

Example: $5/2 = 2$ and 1 is remainder. We can write it in this form: $5\%2$

Q.17: DEFINE ALL ARITHMETIC OPERATORS WITH EXAMPLE.**DEVELOP A SIMPLE CALCULATOR IN C++ BY USING ARITHMETIC OPERATORS.**

```
#include<iostream>
#include<stdio.h>
using namespace std;
int main(void)
{
    int a, b, add, sub, multi, remd;
    float div;
    cout << "\n CALCULATOR";
    cout << "\n =====";
    cout << "\n \t Enter the Value of a.....";
    cin >> a;
    cout << "\n \t Enter the value of b.....";
    cin >> b;
    add = a + b;
    cout << "\n \t Addition of " << a << " and.. " << b << " is...." << add;
    sub = a - b;
    cout << "\n \t Subtraction of " << a << " and.. " << b << " is...." << sub;
    multi = a * b;
    cout << "\n \t Multiplication of " << a << " and.. " << b << " is...." << multi;
    div = a / b;
    cout << "\n \t Division of " << a << " and.. " << b << " is...." << div;
    remd = a % b;
    cout << "\n \t Remainder of Modulus division of " << a << " and.. " << b << " is...." << remd;
    return 0;
}
```

OUTPUT**CALCULATOR**

=====

Enter the Value of a..... 15

Enter the Value of b..... 10

Addition of 15 and 10 is 25

Subtraction of 15 and 10 is 5

Multiplication of 15 and 10 is 150

Division of 15 and 10 is..... 1

Remainder of Modulus division of 15 and 10 is.... 5

Q.18: DEFINE INCREMENT OPERATOR WITH EXAMPLE IN C++.**INCREMENT OPERATORS****Increment Operator (++)**

The increment operator can be used with any type of variable. It is used to add 1 to the value of a variable. The increment operator is represented by ++ (double plus sign). This operator can be applied only to a single variable. There are two ways to use the increment operator:

• **Prefix Increment Operator:** You can apply this operator before the variable name. It can be written like ++a.

i.e., x = ++a;

Example:

```
#include<iostream>
using namespace std;
int main(void)
{
    int a = 10;
    cout << "\n Value of a is....." << ++a;
    return 0;
}
```

OUTPUT

Value of a is 11

In prefix increment operation, the value of a is printed as 11 because 1 is added in a before printing.

Postfix Increment Operator:

If the increment operator is applied after the variable name, it is known as the Postfix Increment operator. It is written like a++.

Example:

```
#include<iostream>
using namespace std;
int main(void)
{
    int a = 10;
    cout << "\n Value of a is....." << a++;
    return 0;
}
```

OUTPUT

Value of a is 10

In Postfix increment operation, the value of a is printed as 10 because 1 is added in a after printing. So, the value of a will change to 11 after printing.

Q.19: DEFINE DECREMENT OPERATOR IN C++.

DECREMENT OPERATORS

C++ also provides the unary decrement operator. It is used to decrement a variable by 1. The decrement operator is represented by -- (double minus sign). The decrement operators are used in two ways (postfix & prefix) summarized below:

Operators	Explanation
--a (Prefix)	Decrement a by 1, then use the new value of a in the expression in which a resides.
a-- (Postfix)	Use the current value of a in the expression in which it resides, then decrement a by 1.

Q.20: DEFINE RELATIONAL OPERATORS WITH EXAMPLE IN C++.

RELATIONAL OPERATORS

Relational operators are used when we have to make comparisons. It is used to test the relation between two values. The result of comparison is True (1) or False (0). C++ programming offers the following relational operators:

Operator	Operations	Example
<	It checks the value on left is less than value on right.	a < b
>	It checks the value on left is greater than value on right.	a > b
<=	It checks the value on left is less than or equal to value on right.	a <= b
>=	It checks the value on left is greater than or equal to value on right.	a >= b
==	It checks the equality of two values.	a == b
!=	It checks the value on left is not equal to value on right.	a != b

PROGRAM USING RELATIONAL OPERATOR IN C++

```
#include<iostream>
using namespace std;
int main()
{
    int x = 20, y = 10;
    if(x > y)
        cout << "X is greater than Y";
    else
        cout << "Y is greater than X";
    return 0;
}
```

Q.21: DEFINE LOGICAL OPERATORS WITH EXAMPLE IN C++.

LOGICAL OPERATORS

Logical operators are used when more than one conditions are to be tested and based on that result, decisions have to be made. C++ programming offers three logical operators. They are:

Operators	Description	Expression
&&	This operator is called AND. The condition will be true if both expressions are true.	x=10, y=5, z=12 x>y && x<z
	It's known as the OR operator. The condition will be true if any one of the expressions is true.	x=10, y=5, z=12 x>y x>z
!	This operator is called NOT. The condition will be inverted, false becomes true and true becomes false.	x=10, y=5; !(x<y);

PROGRAM USING LOGICAL OPERATORS IN C++

```
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
    int num1 = 30, num2 = 40;
    cout << "Logical Operators Example \n";
    if(num1 <= 40 && num2 >= 40)
    {
        cout << "Num1 is less than and Num2 is greater than or equal to 40 \n";
    }
    if(num1 >= 40 || num2 >= 40)
    {
        cout << "Num1 or Num2 is greater than or equal to 40 \n";
    }
    getch();
    return 0;
}
```

OUTPUT

Logical Operators Example

Num1 is less than and Num2 is greater than or equal to 40

Num1 or Num2 is greater than or equal to 40

Q.22: WRITE DOWN THE DIFFERENCE BETWEEN RELATIONAL AND LOGICAL OPERATORS.**DIFFERENTIATE BETWEEN RELATIONAL OPERATOR AND LOGICAL OPERATOR****RELATIONAL OPERATOR**

- Relational operators compare any values in the form of expressions.
- Relational operators are binary operators because they require two operands to operate.
- Relational operators return results either 1 (TRUE) or 0 (FALSE).

LOGICAL OPERATOR

- Logical operators perform logical operations on boolean values 1 (TRUE) and 0 (FALSE).
- Logical operators are usually used to compare one or more relational expressions.
- Logical operators also return output as 1 (TRUE) or 0 (FALSE).

Q.23: DEFINE ASSIGNMENT OPERATOR.**ASSIGNMENT OPERATOR**

Assignment operator (=) are used to assign result of an expression or a value to a variable. The associativity of assignment operators is right to left means value or expression at the right is assigned to the left side variable.

Q24: DEFINE ARITHMETIC ASSIGNMENT OPERATOR.**ARITHMETIC ASSIGNMENT OPERATOR**

Arithmetic assignment operator is a combination of arithmetic. and assignment operators. This operator first performs an arithmetic operation on the current value of the variable on left to the value on the right and then assigns the result to the variable on the left.

Operators	Description
+= (Addition assignment)	It adds the right operand to the left operand and assigns the result to the left operand. Example: $a += 2$ means $a = a + 2$
-= (Subtraction assignment)	It subtracts the right operand from the left operand and assigns the result to the left operand. Example: $a -= 3$ means $a = a - 3$
*= (Multiplication assignment)	It multiplies the right operand with the left operand and assigns the result to the left operand. Example: $a *= 4$ means $a = a * 4$
/= (Division-assignment)	It divides the left operand with the right operand and assigns the result to the left operand. Example: $a /= 4$ means $a = a / 4$